

Project Report

Database Management Systems

(Course code: 25TU03MJM0)



Submitted by:

Syed Mohammed Awais Khadiri

ID: 25WU0102283

Specialization: Artificial Intelligence and Machine Learning

Section: Whales

Faculty: Dr. Kiran Mayee Adavala

(B. Tech 2025-29)

School of Technology

Submitted on: 07/10/2026

Design and Implementation of a Database Management System for Municipal Waste Collection and Recycling Management System

Abstract

Municipal waste management involves several connected activities: residents generate waste, collection vehicles and drivers collect it, routes organize daily operations, recycling centers receive material, and complaints provide a way to report service issues. When these activities are managed manually, the same problems repeat: resident and area information becomes inconsistent, collection records are difficult to track, vehicle capacity may be exceeded, processing quantities may not match collected waste, and complaints are difficult to monitor.

This project presents the design and implementation of a relational database system, the **Municipal Waste Collection and Recycling Management System**, that addresses these problems. The database is built in MySQL and organized into 10 related tables: Area, Resident, Waste_Type, Driver, Vehicle, Route, Waste_Collection, Recycling_Center, Waste_Processing and Complaint. The schema is designed around primary keys, foreign keys and normalization to Third Normal Form (3NF). Database triggers enforce important business rules such as positive collection quantity, vehicle capacity limits, valid processing quantities, valid complaint dates and prevention of processing for missed collections.

The database is populated with realistic sample data and supports reporting through SQL joins, aggregation, nested queries, NOT EXISTS logic, CASE expressions and five views. Reports cover residents and waste by area, waste-type totals, driver performance, collection status, vehicle usage, recycling-center processing and complaint-related service issues. A web-based user interface built with Python Flask and mysql-connector-python connects to the database and supports viewing, searching, insertion and deletion of resident records, with edit functionality added to demonstrate full CRUD operations.

Keywords: DBMS, MySQL, normalization, 3NF, SQL views, Flask, municipal waste management, recycling, CRUD.

Table of Contents

1. Introduction and Problem Statement
 - 1.1 Introduction
 - 1.2 Problem statement
2. Objectives and Scope
 - 2.1 Objectives
 - 2.2 Users of the system
 - 2.3 Scope
3. Software and Hardware Requirements
 - 3.1 Software requirements
 - 3.2 Hardware requirements
4. ER Diagram
 - 4.1 Key relationships
5. Relational Schema and Normalization
 - 5.1 Relational schema
 - 5.2 Normalization to Third Normal Form
6. Data Dictionary
 - 6.1 Area
 - 6.2 Resident
 - 6.3 Waste_Type
 - 6.4 Driver
 - 6.5 Vehicle
 - 6.6 Route
 - 6.7 Waste_Collection
 - 6.8 Recycling_Center
 - 6.9 Waste_Processing
 - 6.10 Complaint
7. SQL Commands Used (DDL and DML)
 - 7.1 DDL: creating the database and tables
 - 7.2 DML: inserting data
 - 7.3 Sample data summary
 - 7.4 Sample output
8. Queries with Outputs
 - 8.1 Area waste summary
 - 8.2 Above-average waste types
 - 8.3 Driver performance
 - 8.4 Top residents by waste
 - 8.5 Areas with delayed or missed collection
 - 8.6 Collection status percentage
 - 8.7 Unused vehicles
 - 8.8 Recycling centers above average recyclable waste
 - 8.9 Residents with complaints and service issues
 - 8.10 Driver performance summary
 - 8.11 Query given during Presentation-II
9. UI Design and Screenshots

- 9.1 Dashboard
- 9.2 Resident Management
- 9.3 Resident CRUD Operations
- 9.4 Operational Management Screens
- 10. Implementation Details
 - 10.1 Technology stack
 - 10.2 Database connectivity
 - 10.3 Insert, update and delete
 - 10.4 Reports and trigger handling
 - 10.5 How to run
- 11. Testing
- 12. Conclusion and Future Enhancements
 - 12.1 Conclusion
 - 12.2 Future enhancements
- Acknowledgement
- References
- Appendix: GitHub Repository Link

1. Introduction and Problem Statement

1.1 Introduction

Municipal waste management requires coordination between residents, collection vehicles, drivers, routes, waste categories, processing facilities and complaint handling. Every collection creates operational information that must remain connected to the resident, vehicle and type of waste. Tracking these records manually or in separate spreadsheets can create duplicate information, missing records and difficulty in generating reliable reports.

A database management system provides a single consistent store for this information. By modelling areas, residents, waste collection, vehicles, routes and processing as related tables, the system keeps the data organized and allows database constraints and triggers to reject invalid records before they affect reports.

1.2 Problem statement

- **Collection tracking:** missed and delayed collections must be clearly identified and reported.
- **Capacity violations:** a collection should not record more waste than the assigned vehicle can carry.
- **Processing mismatch:** recyclable, organic and non-recyclable quantities must not exceed the collected quantity.
- **Complaint monitoring:** complaints need resident linkage and valid dates and statuses.
- **Scattered records:** residents, routes, vehicles and recycling centers must be connected for useful reports.

The goal of this project is to design and implement a normalized relational database with a working Flask user interface that manages municipal waste collection and recycling information from resident registration through collection, processing, reporting and complaint handling.

2. Objectives and Scope

2.1 Objectives

- Maintain a centralized database for municipal waste management records.
- Maintain relationships between residents and their areas.
- Assign drivers to vehicles and vehicles to collection routes.
- Record waste collections with date, quantity, waste type and status.
- Enforce vehicle-capacity and positive-quantity rules through database triggers.
- Ensure processing quantities do not exceed the original collected quantity.
- Track complaints and prevent future-dated complaint records.
- Generate operational reports using joins, aggregation, nested queries and views.
- Provide a usable Flask interface for viewing, searching and modifying resident records.

2.2 Users of the system

User	Role in the system
Admin / Municipal Coordinator	Maintains residents, routes, vehicles, collection records and reports.
Collection Staff	Uses vehicle, driver and route information for collection operations.
Recycling Center Staff	Uses processing records to track recyclable, organic and non-recyclable material.
Resident / Public User	Is represented in the database and may have collection and complaint records.

2.3 Scope

The system covers area and resident management, waste-type classification, driver and vehicle assignment, route scheduling, waste collection, recycling centers, waste processing, complaints and reporting. The Flask UI currently demonstrates dashboard viewing, global search and resident CRUD operations. Out of scope for this version are user login and role-based access, live GPS tracking, online complaint submission from a public portal, automated SMS/e-mail notifications and route optimization.

3. Software and Hardware Requirements

3.1 Software requirements

Component	Details
Database	MySQL 9.7.1 / MySQL Community Server
Database tool	MySQL Workbench and MySQL command-line client
Programming language	Python 3.13.14
Web framework	Flask 3.1.3
DB connector	mysql-connector-python 26.7.0
Front end	HTML and CSS rendered through Flask templates
Browser	Any modern web browser
Code editor	VS Code or any text editor
Operating system	Windows 10/11; project developed on Windows

3.2 Hardware requirements

Component	Minimum requirement
Processor	Dual-core processor, 2 GHz or higher
Memory (RAM)	4 GB or more
Storage	At least 1 GB free disk space
Display	1366 x 768 resolution or higher
Network	Not required for local Flask/MySQL execution

4. ER Diagram

4.1 Key relationships

No.	Relationship	Meaning
1	Area (1) to Resident (M)	One area can contain many residents.
2	Area (1) to Route (M)	One area can have multiple collection routes.
3	Driver (1) to Vehicle (M)	A driver is linked to one or more vehicle records in the current schema.
4	Vehicle (1) to Route (M)	A vehicle can be referenced by route records.
5	Resident (1) to Waste_Collection (M)	A resident can have many collection records.
6	Vehicle (1) to Waste_Collection (M)	A vehicle can perform many collections.
7	Waste_Type (1) to Waste_Collection (M)	One waste type can classify many collections.
8	Waste_Collection (1) to Waste_Processing (M)	A collection can have processing records.
9	Recycling_Center (1) to Waste_Processing (M)	A center can process many processing records.
10	Resident (1) to Complaint (M)	A resident can submit multiple complaints.

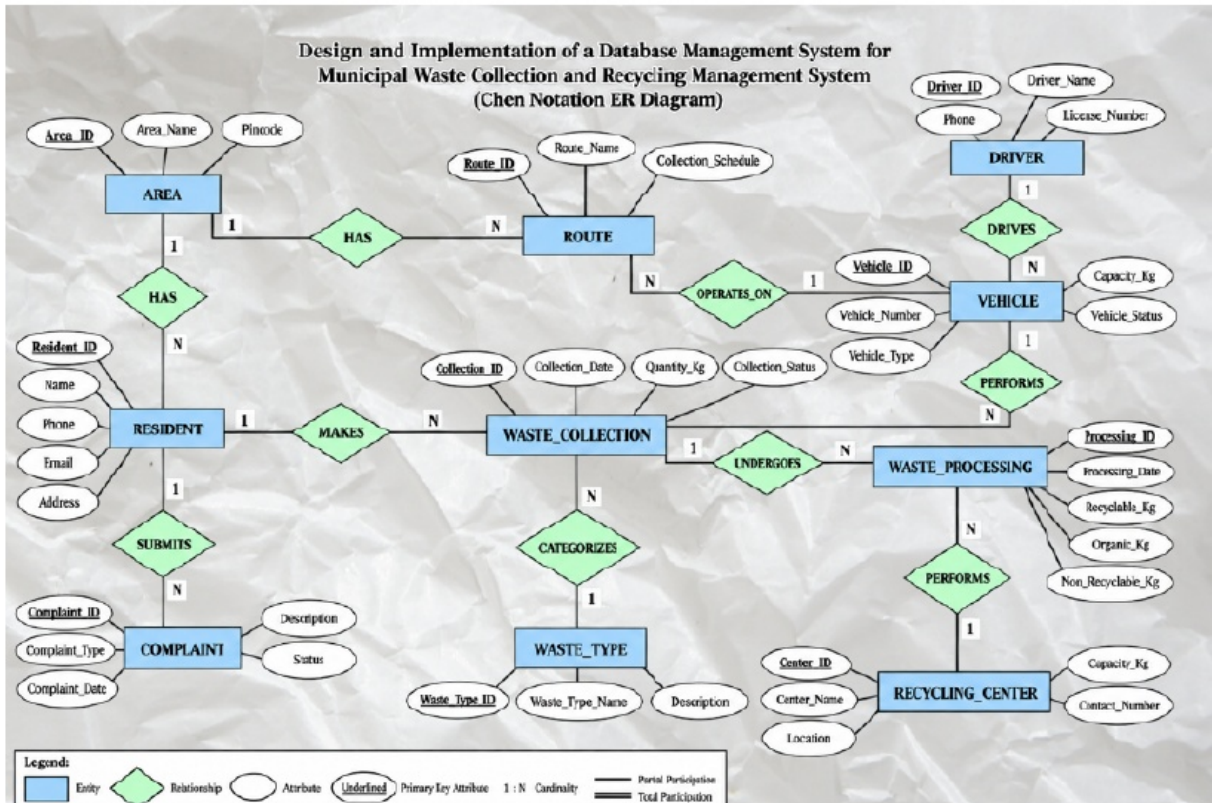


Figure 4.1: Entity-relationship diagram (Chen notation)

5. Relational Schema and Normalization

5.1 Relational schema

Primary keys are the first attribute of each table. Foreign keys are marked (FK).

Table	Schema
Area	(Area_ID, Area_Name, Pincode)
Resident	(Resident_ID, Name, Phone, Email, Address, Area_ID FK)
Waste_Type	(Waste_Type_ID, Waste_Type_Name, Description)
Driver	(Driver_ID, Driver_Name, Phone, License_Number)
Vehicle	(Vehicle_ID, Vehicle_Number, Vehicle_Type, Capacity_Kg, Driver_ID FK, Vehicle_Status)
Route	(Route_ID, Route_Name, Area_ID FK, Vehicle_ID FK, Collection_Schedule)
Waste_Collection	(Collection_ID, Resident_ID FK, Vehicle_ID FK, Collection_Date, Waste_Type_ID FK, Quantity_Kg, Collection_Status)
Recycling_Center	(Center_ID, Center_Name, Location, Capacity_Kg, Contact_Number)
Waste_Processing	(Processing_ID, Collection_ID FK, Center_ID FK, Processing_Date, Recyclable_Kg, Organic_Kg, Non_Recyclable_Kg)
Complaint	(Complaint_ID, Resident_ID FK, Complaint_Date, Complaint_Type, Description, Status)

5.2 Normalization to Third Normal Form

First Normal Form (1NF). Every column contains an atomic value and there are no repeating groups. For example, a resident's phone, e-mail and address are stored as separate attributes, while multiple collections are stored as separate rows in Waste_Collection.

Second Normal Form (2NF). The tables use single-column surrogate primary keys, so non-key attributes depend on the whole primary key. The foreign keys connect related tables rather than repeating descriptive data.

Third Normal Form (3NF). Non-key attributes depend on the key of their own table and not on another non-key attribute. For example, Area_Name and Pincode are stored in Area and referenced by Area_ID, while Driver_Name is stored in Driver and referenced through Vehicle.Driver_ID.

Design note. Reporting totals such as area waste, driver performance and collection-status percentages are derived by queries and views instead of being duplicated as stored summary columns.

6. Data Dictionary

Table structure, data types and constraints for all 10 tables, taken from the DDL design. PK = primary key, FK = foreign key, AI = auto increment.

6.1 Area

Column	Data type	Constraints
Area_ID	INT	PK, AI
Area_Name	VARCHAR(100)	NOT NULL
Pincode	VARCHAR(10)	NOT NULL

6.2 Resident

Column	Data type	Constraints
Resident_ID	INT	PK, AI
Name	VARCHAR(100)	NOT NULL
Phone	VARCHAR(15)	NULL allowed
Email	VARCHAR(100)	NULL allowed
Address	VARCHAR(255)	NULL allowed
Area_ID	INT	FK to Area

6.3 Waste_Type

Column	Data type	Constraints
Waste_Type_ID	INT	PK, AI
Waste_Type_Name	VARCHAR(50)	NOT NULL
Description	VARCHAR(255)	NULL allowed

6.4 Driver

Column	Data type	Constraints
Driver_ID	INT	PK, AI
Driver_Name	VARCHAR(100)	NOT NULL
Phone	VARCHAR(15)	NULL allowed
License_Number	VARCHAR(50)	UNIQUE

6.5 Vehicle

Column	Data type	Constraints
Vehicle_ID	INT	PK, AI
Vehicle_Number	VARCHAR(20)	NOT NULL, UNIQUE
Vehicle_Type	VARCHAR(50)	NULL allowed
Capacity_Kg	DECIMAL(10,2)	NULL allowed
Driver_ID	INT	FK to Driver
Vehicle_Status	VARCHAR(30)	NULL allowed

6.6 Route

Column	Data type	Constraints
Route_ID	INT	PK, AI
Route_Name	VARCHAR(100)	NOT NULL
Area_ID	INT	FK to Area
Vehicle_ID	INT	FK to Vehicle
Collection_Schedule	VARCHAR(100)	NULL allowed

6.7 Waste_Collection

Column	Data type	Constraints
Collection_ID	INT	PK, AI
Resident_ID	INT	FK to Resident
Vehicle_ID	INT	FK to Vehicle
Collection_Date	DATE	NOT NULL
Waste_Type_ID	INT	FK to Waste_Type
Quantity_Kg	DECIMAL(10,2)	NOT NULL
Collection_Status	VARCHAR(30)	NULL allowed

6.8 Recycling_Center

Column	Data type	Constraints
Center_ID	INT	PK, AI
Center_Name	VARCHAR(100)	NOT NULL
Location	VARCHAR(255)	NULL allowed
Capacity_Kg	DECIMAL(10,2)	NULL allowed
Contact_Number	VARCHAR(15)	NULL allowed

6.9 Waste_Processing

Column	Data type	Constraints
Processing_ID	INT	PK, AI
Collection_ID	INT	FK to Waste_Collection
Center_ID	INT	FK to Recycling_Center
Processing_Date	DATE	NOT NULL
Recyclable_Kg	DECIMAL(10,2)	NULL allowed
Organic_Kg	DECIMAL(10,2)	NULL allowed
Non_Recyclable_Kg	DECIMAL(10,2)	NULL allowed

6.10 Complaint

Column	Data type	Constraints
Complaint_ID	INT	PK, AI
Resident_ID	INT	FK to Resident
Complaint_Date	DATE	NOT NULL
Complaint_Type	VARCHAR(100)	NULL allowed
Description	VARCHAR(500)	NULL allowed
Status	VARCHAR(30)	NULL allowed

Indexes and integrity rules

Primary keys and foreign keys provide the main integrity structure. The Driver license number and Vehicle number are unique. The application also uses five database triggers to enforce operational rules.

Triggers

Trigger	Purpose
check_waste_quantity	Rejects a new collection when Quantity_Kg is less than or equal to zero.
check_vehicle_capacity	Rejects a collection when Quantity_Kg exceeds the assigned vehicle capacity.
check_processing_quantity	Rejects processing when recyclable + organic + non-recyclable quantity exceeds the original collection quantity.
check_complaint_date	Rejects a complaint dated in the future.
prevent_missed_collection_processing	Rejects processing for a collection whose status is Missed.

Database design summary

Item	Value
Tables	10
Sample records	300 initial records (30 per table)
Primary keys	10
Foreign-key relationships	Multiple cross-table relationships
Triggers	5
Views	5
Flask application	Python + Flask + mysql-connector-python

7. SQL Commands Used (DDL and DML)

7.1 DDL: creating the database and tables

The complete script is stored in 01_DDL.sql. Representative commands:

```
CREATE DATABASE municipal_waste_management;
USE municipal_waste_management;

CREATE TABLE Area (
    Area_ID INT PRIMARY KEY AUTO_INCREMENT,
    Area_Name VARCHAR(100) NOT NULL,
    Pincode VARCHAR(10) NOT NULL
);

CREATE TABLE Resident (
    Resident_ID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15),
    Email VARCHAR(100),
    Address VARCHAR(255),
    Area_ID INT,
    FOREIGN KEY (Area_ID) REFERENCES Area(Area_ID)
);
```

7.2 DML: inserting data

The complete sample-data script is stored in 02_SampleData.sql. The database was populated with 30 records for each of the 10 tables.

```
INSERT INTO Area (Area_Name, Pincode)
VALUES ('Miyapur', '500049');

INSERT INTO Resident
(Name, Phone, Email, Address, Area_ID)
VALUES ('Aarav Sharma', '9000000001',
        'aarav@example.com', 'Miyapur', 1);
```

7.3 Sample data summary

Table	Rows
Area	30
Resident	30+
Waste_Type	30
Driver	30
Vehicle	30
Route	30
Waste_Collection	30
Recycling_Center	30
Waste_Processing	30
Complaint	30

7.4 Sample output

SHOW TABLES confirms the ten database tables. SELECT queries were used after insertion to verify the records and relationships.

8. Queries with Outputs

The full set is stored in 03_Queries_and_Views.sql. The following queries demonstrate joins, aggregation, nested logic and reporting on the municipal waste dataset.

8.1 Area waste summary (join and aggregation)

```
SELECT a.Area_Name,
       COUNT(DISTINCT r.Resident_ID) AS residents,
       COALESCE(SUM(wc.Quantity_Kg),0) AS total_waste_kg
FROM Area a
LEFT JOIN Resident r ON r.Area_ID = a.Area_ID
LEFT JOIN Waste_Collection wc
      ON wc.Resident_ID = r.Resident_ID
GROUP BY a.Area_ID, a.Area_Name
ORDER BY total_waste_kg DESC;
```

Area_Name	residents	total_waste_kg
Banjara Hills	sample rows	calculated by MySQL
Tarnaka	sample rows	calculated by MySQL
Miyapur	sample rows	calculated by MySQL

8.2 Waste types above average total quantity

```
SELECT wt.Waste_Type_Name,
       SUM(wc.Quantity_Kg) AS total_kg
FROM Waste_Type wt
JOIN Waste_Collection wc
      ON wt.Waste_Type_ID = wc.Waste_Type_ID
GROUP BY wt.Waste_Type_ID, wt.Waste_Type_Name
HAVING SUM(wc.Quantity_Kg) >
       (SELECT AVG(total_kg)
        FROM (SELECT SUM(Quantity_Kg) AS total_kg
              FROM Waste_Collection
              GROUP BY Waste_Type_ID) x);
```

This nested aggregation compares each waste type with the average total quantity across waste types that occur in the collection data.

8.3 Driver performance (above-average handled waste)

```
SELECT d.Driver_Name,
       SUM(wc.Quantity_Kg) AS handled_kg
FROM Driver d
JOIN Vehicle v ON v.Driver_ID = d.Driver_ID
JOIN Waste_Collection wc ON wc.Vehicle_ID = v.Vehicle_ID
GROUP BY d.Driver_ID, d.Driver_Name
HAVING SUM(wc.Quantity_Kg) >
       (SELECT AVG(driver_total)
        FROM (SELECT SUM(Quantity_Kg) AS driver_total
              FROM Waste_Collection
              GROUP BY Vehicle_ID) x)
ORDER BY handled_kg DESC;
```

8.4 Top residents by waste quantity

```
SELECT r.Name,
       a.Area_Name,
       SUM(wc.Quantity_Kg) AS total_waste_kg
FROM Resident r
JOIN Area a ON a.Area_ID = r.Area_ID
JOIN Waste_Collection wc
  ON wc.Resident_ID = r.Resident_ID
GROUP BY r.Resident_ID, r.Name, a.Area_Name
ORDER BY total_waste_kg DESC
LIMIT 5;
```

Resident	Area	Total waste
Top resident	Area returned by MySQL	Calculated
Second resident	Area returned by MySQL	Calculated
Third resident	Area returned by MySQL	Calculated
Fourth resident	Area returned by MySQL	Calculated
Fifth resident	Area returned by MySQL	Calculated

8.5 Areas with delayed or missed collection

```
SELECT DISTINCT a.Area_Name
FROM Area a
JOIN Resident r ON r.Area_ID = a.Area_ID
JOIN Waste_Collection wc ON wc.Resident_ID = r.Resident_ID
WHERE wc.Collection_Status IN ('Delayed', 'Missed')
ORDER BY a.Area_Name;
```

8.6 Collection status percentage

```
SELECT Collection_Status,
       COUNT(*) AS total,
       ROUND(100.0 * COUNT(*) /
             (SELECT COUNT(*) FROM Waste_Collection), 2)
       AS percentage
FROM Waste_Collection
GROUP BY Collection_Status;
```

Collection_Status	Total	Percentage
Collected	24	80.00% (sample data)
Delayed	4	13.33% (sample data)
Missed	2	6.67% (sample data)

8.7 Vehicles unused in collections (NOT EXISTS)

```
SELECT v.Vehicle_ID, v.Vehicle_Number,
       v.Vehicle_Status
FROM Vehicle v
WHERE NOT EXISTS (
  SELECT 1
  FROM Waste_Collection wc
  WHERE wc.Vehicle_ID = v.Vehicle_ID
)
ORDER BY v.Vehicle_ID;
```

The NOT EXISTS condition identifies vehicles that currently have no matching collection record.

8.8 Recycling centers above average recyclable waste

```

SELECT rc.Center_Name,
       SUM(wp.Recyclable_Kg) AS recyclable_kg
FROM Recycling_Center rc
JOIN Waste_Processing wp
  ON wp.Center_ID = rc.Center_ID
GROUP BY rc.Center_ID, rc.Center_Name
HAVING SUM(wp.Recyclable_Kg) >
       (SELECT AVG(center_total)
        FROM (SELECT SUM(Recyclable_Kg) AS center_total
              FROM Waste_Processing
              GROUP BY Center_ID) x)
ORDER BY recyclable_kg DESC;

```

8.9 Residents with complaints and delayed/missed collections

```

SELECT DISTINCT r.Name,
               a.Area_Name
FROM Resident r
JOIN Area a ON a.Area_ID = r.Area_ID
JOIN Complaint c ON c.Resident_ID = r.Resident_ID
JOIN Waste_Collection wc
  ON wc.Resident_ID = r.Resident_ID
WHERE wc.Collection_Status IN ('Delayed', 'Missed')
ORDER BY r.Name;

```

This query combines complaint and collection records to identify residents who experienced a service issue and also raised a complaint.

8.10 Driver performance summary using CASE

```

SELECT d.Driver_Name,
       COUNT(wc.Collection_ID) AS collections,
       COALESCE(SUM(wc.Quantity_Kg),0) AS total_kg,
       SUM(CASE WHEN wc.Collection_Status='Delayed'
                THEN 1 ELSE 0 END) AS delayed,
       SUM(CASE WHEN wc.Collection_Status='Missed'
                THEN 1 ELSE 0 END) AS missed
FROM Driver d
JOIN Vehicle v ON v.Driver_ID = d.Driver_ID
LEFT JOIN Waste_Collection wc
  ON wc.Vehicle_ID = v.Vehicle_ID
GROUP BY d.Driver_ID, d.Driver_Name
ORDER BY total_kg DESC;

```

8.11 Query given during Presentation-II

For the resident Zaid Hussain, display the Area ID, type of waste collected, driver name, vehicle type, and collection status.

```

SELECT r.Name AS Resident_Name,
       a.Area_ID,
       wt.Waste_Type_Name AS Waste,
       d.Driver_Name,
       v.Vehicle_Type,
       wc.Collection_Status
FROM Resident r
JOIN Area a ON r.Area_ID = a.Area_ID
JOIN Waste_Collection wc ON r.Resident_ID = wc.Resident_ID
JOIN Waste_Type wt ON wc.Waste_Type_ID = wt.Waste_Type_ID
JOIN Vehicle v ON wc.Vehicle_ID = v.Vehicle_ID
JOIN Driver d ON v.Driver_ID = d.Driver_ID
WHERE r.Name = 'Zaid Hussain';

```

9. UI Design and Screenshots

The following figures are screenshots captured from the implemented Flask web interface.

9.1 Dashboard



Figure 9.1: Municipal Waste Management Dashboard

9.2 Resident Management

Municipal Waste Management

Residents

View and search registered residents.

Search by name, phone, email or area...

+ Add New Resident

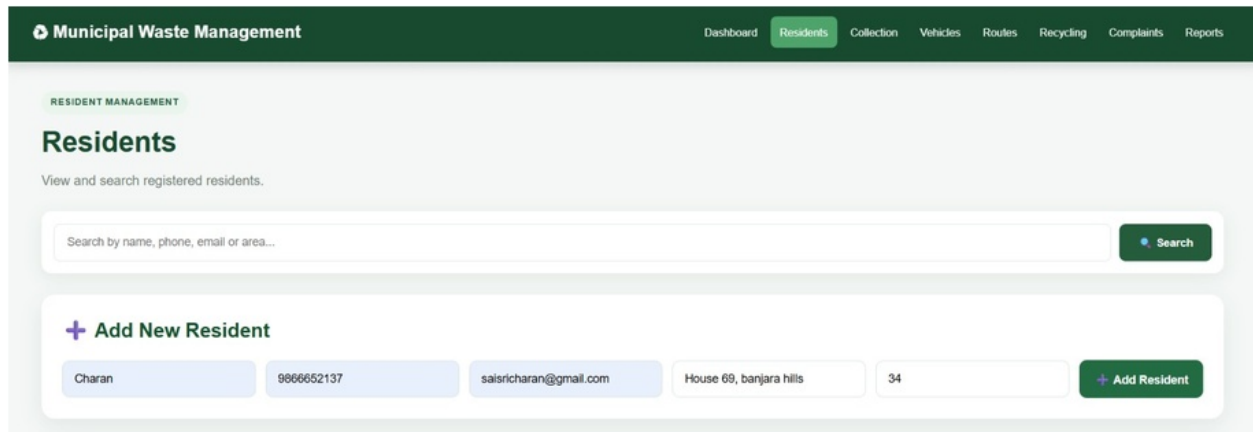
Charan 9866652137 saishicharan@gmail.com House 49, Bangalore hills 34 + Add Resident

Figure 9.2: Residents page with Add New Resident form

9.3 Resident CRUD Operations

The screenshots below demonstrate insertion and deletion of a resident record.

Before Insert



Municipal Waste Management

Dashboard Residents Collection Vehicles Routes Recycling Complaints Reports

RESIDENT MANAGEMENT

Residents

View and search registered residents.

Search by name, phone, email or area...

+ Add New Resident

Charan 9866652137 sauricharan@gmail.com House 69, banjara hills 34 **+ Add Resident**

Figure 9.3: Before insert — Add New Resident form filled with resident details

After Insert

#22	Pooja Sharma	9876543231	pooja.sharma@gmail.com	Flat 203, Somejguda	Somejguda	510082	Delete
#23	Zaid Hussain	9876543232	zaid.hussain@gmail.com	House 31, Panjagutta	Panjagutta	510082	Delete
#24	Lakshmi Devi	9876543233	lakshmi.devi@gmail.com	House 17, Koli	Koli	510086	Delete
#25	Nikhil Bansal	9876543234	nikhil.bansal@gmail.com	Flat 105, Akola	Akola	510091	Delete
#26	Swathi Rao	9876543235	swathi.rao@gmail.com	House 29, Himayalnagar	Himayalnagar	510029	Delete
#27	Manoj Yadav	9876543236	manoj.yadav@gmail.com	House 03, Choudhurnagar	Choudhurnagar	510050	Delete
#28	Harni Reddy	9876543237	harni.reddy@gmail.com	Flat 302, LR Nagar	LR Nagar	510074	Delete
#29	Sameer Khan	9876543238	sameer.khan@gmail.com	House 21, Uppal	Uppal	510039	Delete
#30	Ishta Das	9876543239	ishta.das@gmail.com	Flat 204, Tanaka	Tanaka	510017	Delete
#34	Charan	9866652137	sauricharan@gmail.com	House 69, banjara hills	Banjara Hills	510054	Delete

Figure 9.4: After insert — resident #34 visible in the table

Before Delete

#22	Pooja Sharma	9876543231	pooja.sharma@gmail.com	Flat 203, Somejguda	Somejguda	510082	Delete
#23	Zaid Hussain	9876543232	zaid.hussain@gmail.com	House 31, Panjagutta	Panjagutta	510082	Delete
#24	Lakshmi Devi	9876543233	lakshmi.devi@gmail.com	House 17, Koli	Koli	510086	Delete
#25	Nikhil Bansal	9876543234	nikhil.bansal@gmail.com	Flat 105, Akola	Akola	510091	Delete
#26	Swathi Rao	9876543235	swathi.rao@gmail.com	House 29, Himayalnagar	Himayalnagar	510029	Delete
#27	Manoj Yadav	9876543236	manoj.yadav@gmail.com	House 03, Choudhurnagar	Choudhurnagar	510050	Delete
#28	Harni Reddy	9876543237	harni.reddy@gmail.com	Flat 302, LR Nagar	LR Nagar	510074	Delete
#29	Sameer Khan	9876543238	sameer.khan@gmail.com	House 21, Uppal	Uppal	510039	Delete
#30	Ishta Das	9876543239	ishta.das@gmail.com	Flat 204, Tanaka	Tanaka	510017	Delete
#34	Charan	9866652137	sauricharan@gmail.com	House 69, banjara hills	Banjara Hills	510054	Delete

Figure 9.5: Before delete — resident #34 visible with Delete action

After Delete

#12	Pooja Sharma	9876543231	poojasharma@gmail.com	Flat 200, Ganga Gada	Ganga Gada	506032	Delete
#13	Zaid Hussain	9876543232	zaid.hussain@gmail.com	House 31, Paragada	Paragada	506032	Delete
#14	Lakshmi Devi	9876543233	lakshmi.devi@gmail.com	House 17, Koli	Koli	506035	Delete
#15	Neha Bansal	9876543234	nehabansal@gmail.com	Flat 100, Akas	Akas	506031	Delete
#16	Savithi Rao	9876543235	savithi.rao@gmail.com	House 26, Himayathagar	Himayathagar	506039	Delete
#17	Manoj Yadav	9876543236	manoj.yadav@gmail.com	House 60, Disha Nagar	Disha Nagar	506039	Delete
#18	Hemant Reddy	9876543237	hemant.reddy@gmail.com	Flat 302, LB Nagar	LB Nagar	506034	Delete
#19	Sameer Khan	9876543238	sameer.khan@gmail.com	House 21, Uppal	Uppal	506039	Delete
#20	Shruti Das	9876543239	shrutidas@gmail.com	Flat 204, Tarnaka	Tarnaka	506017	Delete

Figure 9.6: After delete — resident #34 removed

9.4 Operational Management Screens

Waste Collection

Waste Collection								
Track waste collection activities								
Filter: All Collected Outgoing Mixed								
Collection Records								
30 records								
ID	Resident	Area	Waste Type	Vehicle	Driver	Date	Quantity	Status
#1	Aamir Sharma	Miyapur	Food Waste	TS36EA10M1	Ramesh Kumar	2025-09-01	4.50 kg	✓ Collected
#2	Ashvi Khatri	Kulkarni	Paper Waste	TS36EA10M2	Satish Reddy	2025-09-01	3.20 kg	✓ Collected
#3	Rahul Reddy	Dakshin	Plastic Bottle	TS36EA10M3	Mahesh Vaidya	2025-09-02	2.80 kg	✓ Collected
#4	Savitha Rao	Nizamabad	Garden Waste	TS36EA10M4	Naveen Kumar	2025-09-02	5.40 kg	✓ Collected
#5	Vincent Singh	Pragathi Nagar	Plastic Packaging	TS36EA10M5	Ravi Teja	2025-09-03	3.75 kg	⚠ Outgoing
#6	Ananya Iyer	Chandamangar	Glass	TS36EA10M6	Imran Khan	2025-09-03	6.20 kg	✓ Collected
#7	Mohammed Ali	Nellur	Netel Cans	TS36EA10M7	Arun Kumar	2025-09-04	2.50 kg	✓ Collected
#8	Priya Vaidya	Himayathagar	Textile Waste	TS36EA10M8	Prakash Rao	2025-09-04	4.10 kg	✓ Collected
#9	Rohan Mehta	Kondapur	E-Waste	TS36EA10M9	Vinodh Reddy	2025-09-05	1.80 kg	✓ Collected
#10	Kanya Nair	Nellur	Batteries	TS36EA10M0	Sanjay Singh	2025-09-05	0.90 kg	✓ Collected
#11	Aditya Nair	Qandamangar	Light Bulbs	TS36EA10M1	Rajesh Kumar	2025-09-06	1.20 kg	⚠ Mixed
#12	Fatima Begum	Nizamabad	Used Electronics	TS36EA10M2	Anil Reddy	2025-09-06	2.30 kg	✓ Collected

Figure 9.7: Waste Collection Records

Vehicles and Drivers

VEHICLE MANAGEMENT

Vehicles & Drivers

Manage municipal waste collection vehicles.

Filter: All Active Maintenance Expired

ID	Vehicle Number	Type	Capacity	Driver	Vehicle	Status
#1	TS09CA1001	Bin Type	500-50kg	Ramesh Kumar	00000001	Active
#2	TS09CA1002	Garbage Truck	500-50kg	Suresh Reddy	00000002	Active
#3	TS09CA1003	Composter Truck	800-50kg	Manish Yadav	00000003	Active
#4	TS09CA1004	Bin Type	500-50kg	Naveen Kumar	00000004	Active
#5	TS09CA1005	Garbage Truck	600-50kg	Ravi Teja	00000005	Maintenance
#6	TS09CA1006	Recycling Truck	400-50kg	Imran Khan	00000006	Active
#7	TS09CA1007	Bin Type	500-50kg	Arun Kumar	00000007	Active
#8	TS09CA1008	Composter Truck	500-50kg	Prakash Rao	00000008	Active
#9	TS09CA1009	Garbage Truck	700-50kg	Venkatesh Naidu	00000009	Active
#10	TS09CA1010	Recycling Truck	600-50kg	Sanjay Singh	00000010	Active
#11	TS09CA1011	Bin Type	500-50kg	Deepthi Kumar	00000011	Maintenance
#12	TS09CA1012	Garbage Truck	500-50kg	Anil Reddy	00000012	Active
#13	TS09CA1013	Composter Truck	800-50kg	Mohan Das	00000013	Active
#14	TS09CA1014	Bin Type	500-50kg	Kiran Kumar	00000014	Active

Figure 9.8: Vehicles and Drivers Management

Collection Routes — route management interface showing collection routes, assigned vehicles, drivers, and schedules.

ROUTE MANAGEMENT

Collection Routes

View municipal collection routes and schedules.

ID	Route	Area	Vehicle	Driver	Schedule
#1	Miyapur Residential Route	Miyapur	TS09EA1001	Ramesh Kumar	Daily 06:00 AM
#2	Kukatpally Main Route	Kukatpally	TS09EA1002	Suresh Reddy	Daily 06:30 AM
#3	Bachupally Residential Route	Bachupally	TS09EA1003	Manish Yadav	Daily 07:00 AM
#4	Nizampet Route	Nizampet	TS09EA1004	Naveen Kumar	Daily 08:30 AM
#5	Pragathi Nagar Route	Pragathi Nagar	TS09EA1005	Ravi Teja	Alternate Days 07:00 AM
#6	Chandanagar Route	Chandanagar	TS09EA1006	Imran Khan	Daily 06:00 AM
#7	Madinaguda Route	Madinaguda	TS09EA1007	Arun Kumar	Daily 07:30 AM
#8	Hafeezpet Route	Hafeezpet	TS09EA1008	Prakash Rao	Daily 08:30 AM
#9	Kondapur IT Corridor Route	Kondapur	TS09EA1009	Venkatesh Naidu	Daily 05:30 AM
#10	Madhapur Commercial Route	Madhapur	TS09EA1010	Sanjay Singh	Daily 05:00 AM
#11	Gachibowli Route	Gachibowli	TS09EA1011	Rajesh Kumar	Daily 05:30 AM
#12	Manikonda Route	Manikonda	TS09EA1012	Anil Reddy	Daily 06:30 AM
#13	Narsingi Route	Narsingi	TS09EA1013	Mohan Das	Alternate Days 07:00 AM
#14	Khajaguda Route	Khajaguda	TS09EA1014	Kiran Kumar	Daily 07:00 AM
#15	Jubilee Hills Route	Jubilee Hills	TS09EA1015	Salman Ahmed	Daily 06:00 AM

Figure 9.9: Collection Routes interface

Recycling Centers — recycling management interface showing center locations, capacities, contact details, and capacity indicators.

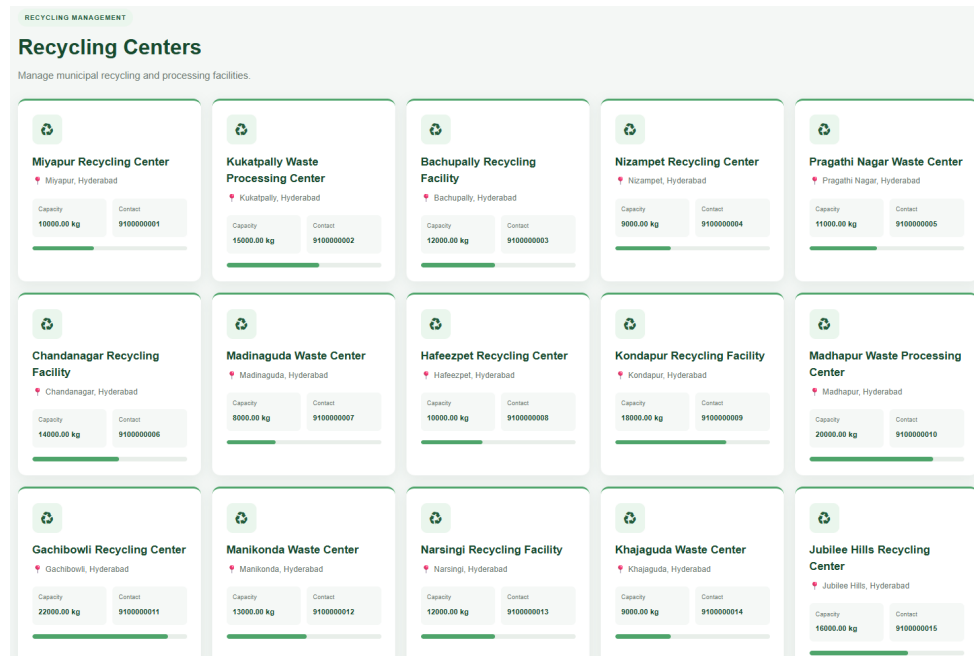


Figure 9.10: Recycling Centers interface

Resident Complaints — complaint management interface showing resident complaints, dates, descriptions, and current status.

COMPLAINT MANAGEMENT						
Resident Complaints						
Monitor waste management complaints and their status.						
Filter: All Pending Resolved Investigation						
ID	Resident	Area	Date	Type	Description	Status
#1	Aarav Sharma	Miyapur	2025-09-01	Missed Collection	Waste was not collected from the household on the scheduled date.	Resolved
#2	Aisha Khan	Kukatpally	2025-09-01	Overflowing Bin	Community waste bin near the residence is completely full.	Resolved
#3	Rahul Reddy	Bachupally	2025-09-02	Delayed Collection	Garbage collection vehicle arrived several hours late.	Resolved
#4	Sneha Rao	Nizampet	2025-09-02	Illegal Dumping	Waste has been dumped illegally near the residential area.	Investigation
#5	Vikram Singh	Pragathi Nagar	2025-09-03	Bad Odor	Strong unpleasant smell is coming from the waste collection point.	Resolved
#6	Ananya Iyer	Chandanagar	2025-09-03	Missed Collection	Organic waste was not collected during the scheduled collection round.	Pending
#7	Mohammed Arif	Madinaguda	2025-09-04	Waste Segregation	Collection staff did not collect segregated waste separately.	Resolved
#8	Priya Verma	Hafeezpet	2025-09-04	Overflowing Bin	Public garbage container is overflowing and attracting stray animals.	Resolved
#9	Rohan Mehta	Kondapur	2025-09-05	Vehicle Issue	Garbage collection vehicle appears to have broken down during collection.	Investigation
#10	Kavya Nair	Madhapur	2025-09-05	Missed Collection	Household waste has remained uncollected for two consecutive days.	Pending
#11	Aditya Kapoor	Gachibowli	2025-09-06	Illegal Dumping	Construction waste has been dumped on the roadside.	Investigation
#12	Fatima Begum	Manikonda	2025-09-06	Delayed Collection	Waste collection was delayed beyond the scheduled time.	Resolved
#13	Sahil Gupta	Narsingi	2025-09-07	Bad Odor	Waste processing area is producing an excessive foul smell.	Resolved
#14	Neha Joshi	Khajaguda	2025-09-07	Overflowing Bin	Public dustbin near the market is overflowing.	Pending

Figure 9.11: Resident Complaints interface

Reports & Analytics — analytics interface showing waste processing totals, collection status, complaint status, and waste-type analysis.

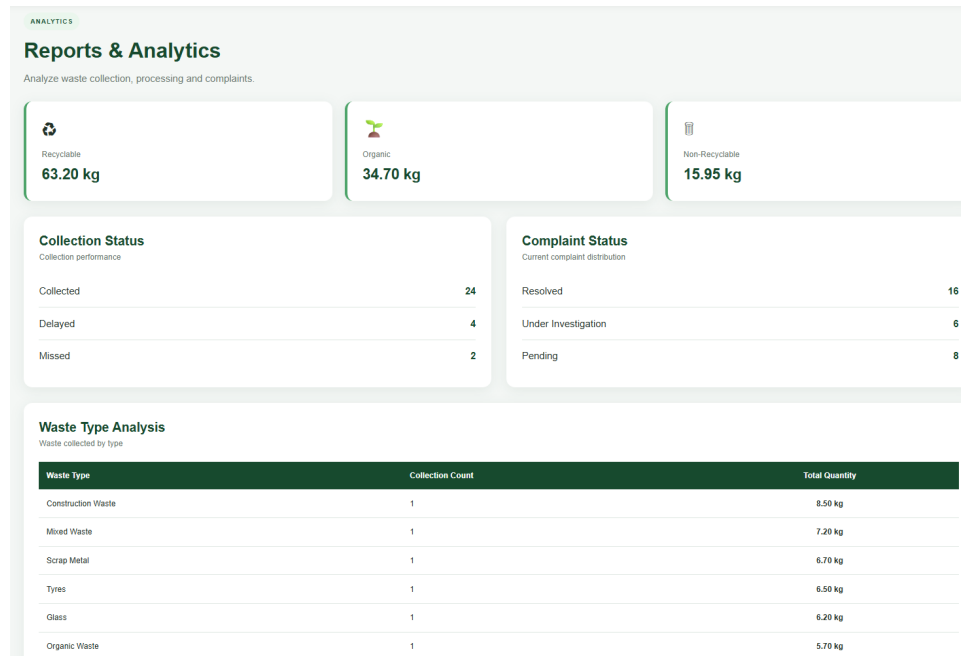


Figure 9.12: Reports & Analytics interface

10. Implementation Details

10.1 Technology stack

Layer	Technology	Role
Presentation	HTML, CSS (Flask templates)	Forms, tables, navigation and messages
Application	Python 3 with Flask	Routes for viewing, searching, insert, update, delete and reports
Connectivity	mysql-connector-python	Opens MySQL connections and executes parameterized SQL
Database	MySQL	Stores records, relationships and trigger-based integrity rules

10.2 Database connectivity

The Flask application opens a MySQL connection through a common `get_db_connection()` function. SQL statements use `%s` parameters for user-provided values, reducing SQL-injection risk. Cursors are closed and database connections are closed after each operation.

```
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "YOUR_MYSQL_PASSWORD",
    "database": "municipal_waste_management",
    "port": 3306
}

def get_db_connection():
    return mysql.connector.connect(**DB_CONFIG)
```

10.3 Insert, update and delete

Resident insertion uses /add_resident. The edit page is served by /edit_resident/<resident_id>, while /update_resident/<resident_id> updates the record. The delete route removes dependent processing, collection and complaint rows before deleting the resident and commits the transaction.

10.4 Reports and trigger handling

The Reports page runs SQL statements that aggregate waste by type, collection status, complaint status, processing quantities and area. Database triggers enforce operational rules independently of the UI. For example, a collection with quantity greater than the vehicle capacity is rejected before insertion.

```
@app.route("/add_resident", methods=["POST"])
def add_resident():
    name = request.form.get("name")
    phone = request.form.get("phone")
    email = request.form.get("email")
    address = request.form.get("address")
    area_id = request.form.get("area_id")

    db = get_db_connection()
    cursor = db.cursor()
    cursor.execute(
        "INSERT INTO Resident "
        "(Name, Phone, Email, Address, Area_ID) "
        "VALUES (%s, %s, %s, %s, %s)",
        (name, phone, email, address, area_id)
    )
    db.commit()
```

10.5 How to run

- Run 01_DDL.sql, 02_SampleData.sql and 03_Queries_and_Views.sql in MySQL.
- Install the packages: pip install flask mysql-connector-python.
- Set the local MySQL password in UI app.py.
- Run python app.py.
- Open http://127.0.0.1:5000 in a browser.

11. Testing

Testing covers normal CRUD operations, foreign-key relationships and trigger-based business rules. The expected results below describe how the database should respond.

ID	Test case	Input	Expected result
T1	View residents	Open Residents	Rows shown from MySQL
T2	Insert valid resident	New resident + valid Area_ID	Row inserted
T3	Edit resident	Change name/area	Updated row shown
T4	Delete resident	Delete a resident	Dependent records removed and resident deleted
T5	Invalid quantity	Quantity <= 0	Rejected by trigger
T6	Capacity violation	Quantity > vehicle capacity	Rejected by trigger
T7	Invalid processing	Processing total > collection	Rejected by trigger
T8	Future complaint	Future Complaint_Date	Rejected by trigger
T9	Missed collection processing	Process a Missed collection	Rejected by trigger
T10	Search resident	Name/phone/area search	Matching results displayed
T11	Collection filter	Status = Delayed	Only delayed rows displayed

ID	Test case	Input	Expected result
T12	Vehicle filter	Status = Active	Active vehicles displayed
T13	Complaint filter	Status = Pending	Pending complaints displayed
T14	Reports page	Open Reports	Aggregated reports load correctly

The testing process verifies both database-level rules and UI-level operations. Because important restrictions are implemented in the database, invalid values remain protected even if an operation is attempted outside the Flask interface.

12. Conclusion and Future Enhancements

12.1 Conclusion

This project delivers a complete normalized MySQL database for municipal waste collection and recycling management, together with a working Flask web interface. The 10-table design connects residents, areas, waste types, drivers, vehicles, routes, collections, recycling centers, processing and complaints. Primary keys and foreign keys maintain relationships, while triggers enforce important operational rules.

The SQL queries and views provide reports for waste quantities, area performance, drivers, collection status, vehicle usage, recycling processing and complaint-related service issues. The UI demonstrates live database connectivity and CRUD operations for residents.

12.2 Future enhancements

- Live GPS tracking for collection vehicles and route progress.
- Automatic route optimization based on waste volume and traffic.
- Resident login and online complaint submission.
- SMS/e-mail notifications for missed or delayed collection.
- Mobile application for collection staff and residents.
- Dashboards with historical trends and predictive waste-generation analytics.
- Role-based access control for administrators, drivers and recycling-center staff.
- Automated scheduling based on vehicle capacity and area demand.

Acknowledgement

I would like to express my sincere gratitude to **Dr. Kiran Mayee Adavala** for the guidance, encouragement, and valuable feedback provided during the development of this individual project, “Municipal Waste Collection and Recycling Management System.” Their support helped me strengthen my understanding of database management concepts and apply them to the planning, design and implementation of the system.

I am grateful for the time and care provided in reviewing my work and offering suggestions throughout the project. This guidance helped me approach the database design thoughtfully, understand normalization and relational modelling, address implementation challenges and improve the quality of the final report.

I also extend my thanks to the faculty and staff of the School of Technology for the academic environment and resources that supported my learning. Finally, I acknowledge everyone whose assistance or encouragement contributed, directly or indirectly, to completing this project.

I accept responsibility for the work presented in this report and for any errors or omissions that may remain.

Syed Mohammed Awais Khadiri

References

- MySQL 8.0/9.x Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- MySQL Connector/Python Developer Guide. Oracle Corporation. <https://dev.mysql.com/doc/connector-python/en/>
- Flask Documentation. Pallets Projects. <https://flask.palletsprojects.com/>
- Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems. Pearson.
- Silberschatz, A., Korth, H. F. and Sudarshan, S. Database System Concepts. McGraw-Hill.
- Course material and instructions, DBMS Course Project, AIML Whales.

Appendix: GitHub Repository Link

Repository: <https://github.com/awais-droid/DBMS-Project.git>

Contents: Presentation-I, Presentation-II, Presentation-III, Project-Report, SQL files, Flask UI source code and screenshots.